

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 1: Introduction to Prisma ORM

## 1.1 What is an ORM?

ORM stands for Object Relational Mapper. It lets you interact with a relational database using your programming language instead of writing raw SQL for every operation.

## 1.2 What is Prisma?

Prisma is a modern ORM for Node.js and TypeScript/JavaScript. It generates a type-safe client from your schema and supports PostgreSQL, MySQL, SQLite, SQL Server and MongoDB (with a different model).

## 1.3 Why Prisma?

- Less boilerplate SQL
- Type-safe queries
- Auto-generated client
- Easy migrations
- Excellent developer experience
- Great integration with Express and Next.js

## 1.4 Prisma in the Stack

Client → Express → Controller → Prisma Client → PostgreSQL

Prisma sits between your application and the database, generating SQL behind the scenes.

## 1.5 SQL vs Prisma

SQL:

```
SELECT * FROM users;
```

Prisma:

```
await prisma.user.findMany();
```

Both retrieve users, but Prisma generates the SQL automatically.

## 1.6 How Prisma Works

1. Define models in schema.prisma.
2. Run migrations.
3. Generate Prisma Client.
4. Import Prisma Client.
5. Use methods such as `findMany()`, `create()`, `update()` and `delete()`.

## 1.7 Advantages

Cleaner code, autocomplete, compile-time checks, easier refactoring and reduced risk of common SQL mistakes.

## 1.8 When to Use Raw SQL

Prisma covers most CRUD operations. For very complex queries or database-specific features, Prisma provides `$queryRaw` and `$executeRaw`.

## Interview Questions

1. What does ORM stand for?
2. What problem does Prisma solve?
3. How is Prisma different from writing raw SQL?
4. Where does Prisma fit in a backend architecture?
5. When would you still use raw SQL?

## Practice Exercises

1. Explain ORM in your own words.
2. Compare SQL and Prisma for a simple SELECT query.
3. List three advantages of Prisma.
4. Draw the request flow including Prisma.
5. Research which databases Prisma supports.

## Chapter Summary

You learned what an ORM is, why Prisma is popular, how it fits into a backend application, how it compares with raw SQL, and when raw SQL is still appropriate.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 2: Installing Prisma & Project Setup

## 2.1 Prerequisites

Before using Prisma, install Node.js and PostgreSQL, create a database, and initialize a Node.js project using `npm init -y`.

## 2.2 Install Prisma

Commands:

```
npm install prisma --save-dev
npm install @prisma/client
```

The prisma package provides the CLI. `@prisma/client` is used inside your application.

## 2.3 Initialize Prisma

Run:

```
npx prisma init
```

This creates:

- `prisma/schema.prisma`
- `.env`

## 2.4 The .env File

`DATABASE_URL` stores your database connection string.

Example:

```
DATABASE_URL="postgresql://postgres:password@localhost:5432/practice"
```

Never commit real credentials to public repositories.

## 2.5 schema.prisma

This file contains:

- `generator`
- `datasource`
- `models`

Initially it includes the `datasource` and `generator` blocks. Models are added later.

## 2.6 Generator Block

Example:

```
generator client {
  provider = "prisma-client-js"
}
```

It tells Prisma to generate the Prisma Client.

## 2.7 Datasource Block

Example:

```
datasource db {  
  provider = "postgresql"  
  url = env("DATABASE_URL")  
}
```

Prisma reads the connection string from .env.

## 2.8 Generate Prisma Client

Run:

```
npx prisma generate
```

This creates the Prisma Client used in your JavaScript or TypeScript code.

## 2.9 Typical Workflow

1. Update schema.prisma
2. Run migrations
3. Generate client
4. Import PrismaClient
5. Query the database

## 2.10 Best Practices

- Keep DATABASE\_URL in .env.
- Use one PrismaClient instance.
- Commit schema.prisma to version control.
- Do not commit secrets.

## Interview Questions

1. Which packages are required to use Prisma?
2. What does npx prisma init do?
3. What is DATABASE\_URL?
4. Why do we run prisma generate?
5. Why should .env not be committed?

## Practice Exercises

1. Install Prisma.
2. Initialize a project.
3. Configure DATABASE\_URL.
4. Run prisma generate.
5. Explain the purpose of schema.prisma.

## Chapter Summary

You learned how to install Prisma, initialize a project, configure the database connection, understand the generator and datasource blocks, and generate the Prisma Client.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 3: Understanding schema.prisma

## 3.1 What is schema.prisma?

schema.prisma is the heart of a Prisma project. It defines the database connection, Prisma Client generator and all database models. Prisma uses this file to generate SQL migrations and the Prisma Client.

## 3.2 File Structure

Typical sections:

- generator
- datasource
- models
- enums (optional)

Every Prisma project revolves around this file.

## 3.3 Generator Block

```
generator client {  
  provider = "prisma-client-js"  
}
```

The generator tells Prisma to generate a type-safe Prisma Client.

## 3.4 Datasource Block

```
datasource db {  
  provider = "postgresql"  
  url = env("DATABASE_URL")  
}
```

The datasource tells Prisma which database is being used and where to find the connection string.

## 3.5 Models

```
model User {  
  id Int @id @default(autoincrement())  
  name String  
  email String @unique  
}
```

Each model usually maps to one database table.

## 3.6 Fields & Scalar Types

Common scalar types:

Int  
String  
Boolean  
Float  
DateTime  
Decimal  
Bytes

Json

Each field becomes a column in the database.

## 3.7 Attributes

Important attributes:

@id - Primary key

@default() - Default value

@unique - Unique constraint

@relation - Defines relationships

@@index - Table-level index

@@unique - Composite unique constraint

## 3.8 Default Values

Examples:

@default(autoincrement())

@default(now())

@default(true)

Defaults are applied when values are not supplied.

## 3.9 Mapping SQL to Prisma

SQL PRIMARY KEY -> @id

AUTO INCREMENT -> @default(autoincrement())

UNIQUE -> @unique

FOREIGN KEY -> @relation

TABLE -> model

## 3.10 Enums

Example:

```
enum Role {  
  USER  
  ADMIN  
}
```

Enums restrict values to a predefined set.

## 3.11 Best Practices

- Keep model names singular.
- Use meaningful field names.
- Add @unique where required.
- Prefer relations over duplicate data.
- Commit schema.prisma to version control.

## Common Mistakes

- Forgetting @id.
- Missing @relation.
- Using the wrong scalar type.
- Changing the schema but forgetting to run a migration.
- Editing the database manually without updating the schema.

## Interview Questions

1. What is schema.prisma?
2. What does the generator block do?
3. What is the datasource block?
4. How does a model relate to a table?
5. Explain @id, @default and @unique.

## Practice Exercises

1. Create a User model.
2. Add a Product model.
3. Add a unique email field.
4. Create a Role enum.
5. Map SQL table columns to Prisma fields.

## Chapter Summary

You learned the structure of schema.prisma, generator and datasource blocks, models, scalar types, attributes, enums, SQL-to-Prisma mapping, best practices and common mistakes.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 4: Models & Migrations

## 4.1 What is a Model?

A Prisma model represents a database table. Each field in the model becomes a column, and each record becomes a row.

## 4.2 Creating a Model

Example:

```
model User {  
  id Int @id @default(autoincrement())  
  name String  
  email String @unique  
}
```

## 4.3 Updating Models

Adding, removing or changing fields in `schema.prisma` changes the database structure. After modifying a model, create a new migration so the database stays in sync.

## 4.4 What is a Migration?

A migration is a version-controlled set of SQL changes generated by Prisma. It updates the database schema safely as your application evolves.

## 4.5 Creating a Migration

Run:

```
npx prisma migrate dev --name init
```

Prisma:

- Compares `schema.prisma` with the database
- Generates SQL migration files
- Applies them
- Regenerates Prisma Client

## 4.6 Migration Files

Migration files contain SQL statements such as `CREATE TABLE`, `ALTER TABLE` and `CREATE INDEX`. They allow every developer to reproduce the same schema.

## 4.7 Drift Detection

If the database schema no longer matches the migration history, Prisma reports drift. This often happens when tables are changed manually or an existing database is connected without matching migrations.

## 4.8 Resetting Development Database

Run:

```
npx prisma migrate reset
```

This drops the development database, reapplies all migrations and regenerates the Prisma Client. Never use it on production data.

## 4.9 Common Migration Workflow

1. Edit schema.prisma
2. Run prisma migrate dev
3. Review generated SQL
4. Test the application
5. Commit schema and migrations

## 4.10 Best Practices

- Never edit migration SQL unless necessary.
- Keep schema.prisma in version control.
- Create small, meaningful migrations.
- Avoid manual schema changes outside Prisma during development.

## Common Mistakes

- Forgetting to run migrations after editing models.
- Ignoring drift warnings.
- Running migrate reset on production.
- Deleting migration folders without understanding the consequences.

## Interview Questions

1. What is a Prisma model?
2. What is a migration?
3. What does prisma migrate dev do?
4. What is drift detection?
5. When should prisma migrate reset be used?

## Practice Exercises

1. Create a Product model.
2. Add a price field to Product.
3. Generate a migration.
4. Inspect the generated SQL.
5. Explain why migrations are committed to Git.

## Chapter Summary

You learned how Prisma models map to database tables, how migrations work, how Prisma detects drift, when to use migrate reset, and the recommended migration workflow.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 5: Prisma Client

## 5.1 What is Prisma Client?

Prisma Client is an auto-generated, type-safe database client. It exposes methods such as `findMany()`, `findUnique()`, `create()`, `update()` and `delete()` based on your schema.

## 5.2 Generating Prisma Client

Run:

```
npx prisma generate
```

Whenever your schema changes, regenerate the client if required so it matches the latest models.

## 5.3 Creating a Single Instance

Example:

```
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();
export default prisma;
```

Create one shared instance and reuse it throughout the application.

## 5.4 Why Only One Instance?

Creating multiple `PrismaClient` instances wastes connections and resources. Import the shared instance from `prisma.js` (or `db/prisma.js`) in controllers and services.

## 5.5 Importing Prisma Client

Example:

```
import prisma from './prisma';
```

Now use `prisma.user.findMany()`, `prisma.product.create()` and other generated methods.

## 5.6 Generated Methods

Common methods:

- `findMany()`
- `findUnique()`
- `create()`
- `update()`
- `delete()`
- `count()`
- `aggregate()`
- `groupBy()`

## 5.7 Basic Query Example

```
const users = await prisma.user.findMany();
```

```
const user = await prisma.user.findUnique({ where: { id: 1 } });
```

## 5.8 Error Handling

Wrap database calls in try...catch. Handle missing records, validation issues and unexpected database errors gracefully.

## 5.9 Best Practices

- Use one shared PrismaClient.
- Keep database logic separate from routes.
- Close connections only for scripts or when appropriate.
- Use async/await consistently.

## Common Mistakes

- Creating PrismaClient inside every request.
- Forgetting await.
- Editing generated client files.
- Using stale generated clients after schema changes.

## Interview Questions

1. What is Prisma Client?
2. Why should there be only one PrismaClient instance?
3. What does prisma generate do?
4. Name five common Prisma Client methods.
5. Where should PrismaClient usually be initialized?

## Practice Exercises

1. Create prisma.js.
2. Export one PrismaClient instance.
3. Import it into a controller.
4. Fetch all users with findMany().
5. Fetch one user with findUnique().

## Chapter Summary

You learned what Prisma Client is, how it is generated, why a single shared instance is recommended, how to import and use it, and the most common generated query methods.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM  
Chapter 6: CRUD Operations

## 6.1 Introduction

Prisma provides simple methods for Create, Read, Update and Delete operations. These methods generate SQL automatically while keeping your code clean and type-safe.

## 6.2 Read - findMany()

Returns multiple records.

Prisma:  
`await prisma.user.findMany();`

SQL:  
`SELECT * FROM users;`

## 6.3 Read - findUnique()

Returns one record using a unique field.

Prisma:  
`await prisma.user.findUnique({ where: { email: 'rahul@gmail.com' } });`

SQL:  
`SELECT * FROM users WHERE email='rahul@gmail.com';`

## 6.4 Create - create()

Creates one record.

Prisma:  
`await prisma.user.create({ data: { name: 'Rahul', email: 'rahul@gmail.com' } });`

SQL:  
`INSERT INTO users(name,email) VALUES('Rahul','rahul@gmail.com');`

## 6.5 Create Many

Prisma:  
`await prisma.user.createMany({ data: [{ name: 'A' }, { name: 'B' }] });`

SQL:  
`INSERT INTO users(name) VALUES('A'),('B');`

## 6.6 Update

Prisma:  
`await prisma.user.update({ where: { id: 1 }, data: { name: 'Aman' } });`

SQL:  
`UPDATE users SET name='Aman' WHERE id=1;`

## 6.7 Update Many

Prisma:

```
await prisma.user.updateMany({ where:{isActive:false}, data:{isActive:true} });
```

## 6.8 Delete

Prisma:

```
await prisma.user.delete({ where:{id:1} });
```

SQL:

```
DELETE FROM users WHERE id=1;
```

## 6.9 Delete Many

Prisma:

```
await prisma.user.deleteMany({ where:{isActive:false} });
```

## 6.10 Controller Example

Flow:

Frontend → Route → Controller → Prisma Client → PostgreSQL → Response

Use try...catch and return proper HTTP status codes.

## 6.11 Best Practices

- Convert route ids to Number when needed.
- Validate request data.
- Use unique fields with findUnique().
- Keep Prisma queries in controllers/services.
- Return meaningful status codes.

## Common Mistakes

- Forgetting where in update/delete.
- Using findUnique on non-unique fields.
- Forgetting await.
- Not handling missing records.

## Interview Questions

1. Difference between findMany() and findUnique()?
2. Difference between update() and updateMany()?
3. Difference between delete() and deleteMany()?
4. Why must findUnique() use a unique field?
5. Match create() with its SQL equivalent.

## Practice Exercises

1. Fetch all users.
2. Fetch one user by email.
3. Create a new product.
4. Update a user's name.
5. Delete a user by id.
6. Write the SQL equivalent of each Prisma query.

## Chapter Summary

You learned Prisma CRUD methods, their SQL equivalents, controller usage, best practices, common mistakes, and interview-focused concepts.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 7: Filtering & Sorting

## 7.1 Why Filtering?

Filtering returns only the records you need instead of the entire table, improving performance and reducing unnecessary data transfer.

## 7.2 where

Prisma uses the where object to filter records.

Prisma:

```
await prisma.user.findMany({ where:{ name:'Nikhil' } });
```

SQL:

```
SELECT * FROM users WHERE name='Nikhil';
```

## 7.3 Comparison Operators

gt (>)

gte (>=)

lt (<)

lte (<=)

Example:

```
where:{ age:{ gte:18 } }
```

## 7.4 String Filters

contains

startsWith

endsWith

Example:

```
where:{ email:{ endsWith:'gmail.com' } }
```

## 7.5 Logical Operators

AND, OR and NOT combine multiple conditions.

Example:

```
where:{ AND:[{isActive:true},{age:{gte:18}}] }
```

## 7.6 orderBy

Sort records.

Prisma:

```
orderBy:{ name:'asc' }
```

SQL:

```
ORDER BY name ASC;
```

## 7.7 Multiple Sorting

Example:

```
orderBy:[{department:'asc'},{salary:'desc'}]
```

Equivalent SQL:

```
ORDER BY department ASC, salary DESC;
```

## 7.8 Combining Filters & Sorting

Prisma:

```
findMany({ where:{ age:{gte:18}}, orderBy:{name:'asc'} })
```

## 7.9 Best Practices

- Filter in the database, not in JavaScript.
- Index frequently searched columns.
- Combine where and orderBy when appropriate.
- Keep queries readable.

## Common Mistakes

- Using findUnique() with non-unique fields.
- Forgetting nested operators.
- Sorting after fetching data unnecessarily.

## Interview Questions

1. What does where do?
2. Difference between gt and gte?
3. Difference between contains and startsWith?
4. How do you sort descending?
5. Can Prisma sort by multiple fields?

## Practice Exercises

1. Find users aged 21 or older.
2. Find emails ending with gmail.com.
3. Sort users by name ascending.
4. Sort employees by department then salary.
5. Combine filtering and sorting in one query.

## Chapter Summary

You learned how to filter data using where, comparison and string operators, combine conditions with AND/OR/NOT, sort with orderBy, and map Prisma queries to SQL.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 8: Pagination

## 8.1 Why Pagination?

Pagination loads only a subset of records instead of the entire table. It improves performance, reduces memory usage and provides a better user experience.

## 8.2 take

take limits the number of records returned.

Prisma:

```
await prisma.user.findMany({ take: 10 });
```

SQL:

```
SELECT * FROM users LIMIT 10;
```

## 8.3 skip

skip ignores a number of records before returning results.

Prisma:

```
await prisma.user.findMany({ skip: 20, take: 10 });
```

SQL:

```
SELECT * FROM users LIMIT 10 OFFSET 20;
```

## 8.4 Offset Pagination

Offset pagination uses skip and take.

Formula:

$\text{skip} = (\text{page} - 1) * \text{limit}$

$\text{take} = \text{limit}$

Example:

Page 3 with 10 records per page:

$\text{skip} = 20$

$\text{take} = 10$

## 8.5 Cursor Pagination

Cursor pagination is more efficient for large datasets.

Prisma:

```
await prisma.user.findMany({  
  cursor: { id: 100 },  
  take: 10,  
  skip: 1  
});
```

It starts after the specified record instead of counting offsets.

## 8.6 Pagination with orderBy

Always combine pagination with a stable order.

```
Prisma:
await prisma.user.findMany({
  orderBy: { id: 'asc' },
  skip: 20,
  take: 10
});
```

## 8.7 SQL ↔ Prisma Mapping

LIMIT -> take  
OFFSET -> skip  
ORDER BY -> orderBy  
Cursor -> cursor

## 8.8 Performance Tips

- Use pagination for large tables.
- Prefer cursor pagination for very large datasets.
- Index columns used for ordering.
- Avoid fetching unnecessary fields.

## Best Practices

- Validate page and limit.
- Provide default values.
- Return pagination metadata.
- Use consistent API responses.

## Common Mistakes

- Using skip without orderBy.
- Returning all records.
- Negative page values.
- Large OFFSET values on huge tables when cursor pagination is more suitable.

## Interview Questions

1. What is the difference between take and skip?
2. How does Prisma implement LIMIT and OFFSET?
3. What is cursor pagination?
4. Why combine pagination with orderBy?
5. When is cursor pagination preferred?

## Practice Exercises

1. Return the first 5 users.
2. Return page 3 with 10 users per page.
3. Add orderBy to a paginated query.
4. Write a cursor-based pagination query.
5. Map Prisma pagination queries to SQL.

## Chapter Summary

You learned offset and cursor pagination, take, skip, cursor, orderBy, SQL-to-Prisma mapping, performance considerations, best practices and interview concepts.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 9: Prisma Relations, select & include

## 9.1 Why Relations?

Relations connect models just as foreign keys connect SQL tables. They allow Prisma to fetch related data without writing JOIN statements manually.

## 9.2 One-to-One

Example: User ↔ Profile.

A user has one profile and a profile belongs to one user.

## 9.3 One-to-Many

Example:

```
model User { id Int @id @default(autoincrement()) orders Order[] }
model Order { id Int @id @default(autoincrement()) userId Int user User
  @relation(fields:[userId], references:[id]) }
```

One user can have many orders.

## 9.4 Many-to-Many

Example: Students ↔ Courses. Prisma can model many-to-many relations using implicit or explicit relation tables.

## 9.5 @relation

@relation defines how two models are connected.

Example:

```
user User @relation(fields:[userId], references:[id])
```

## 9.6 select

select returns only specified fields.

Prisma:

```
await prisma.user.findMany({ select:{ email:true } });
```

SQL:

```
SELECT email FROM users;
```

## 9.7 include

include fetches related records.

Prisma:

```
await prisma.user.findMany({ include:{ orders:true } });
```

Equivalent concept: SQL JOIN.

## 9.8 Nested select

Example:

```
await prisma.user.findMany({ select:{ name:true, orders:{ select:{ product:true }}}});
```

## 9.9 Nested Writes

Create related records together.

Example:

```
create({ data:{ name:'Nikhil', orders:{ create:[{ product:'Laptop' }] }}});
```

## 9.10 SQL vs Prisma

SQL JOINS correspond to Prisma include/select with relations. Prisma generates the JOIN queries internally.

## Best Practices

Use select to limit returned fields. Use include only when related data is needed. Define relations in schema.prisma and keep names meaningful.

## Common Mistakes

Confusing select with include, missing @relation fields, using include for unrelated models, returning unnecessary nested data.

## Interview Questions

1. What does @relation do?
2. Difference between select and include?
3. How is a one-to-many relation defined?
4. How does Prisma replace SQL JOINS?
5. When should you use nested select?

## Practice Exercises

1. Create User and Order models with a one-to-many relation.
2. Return only user emails using select.
3. Return users with orders using include.
4. Return only product names from related orders.
5. Compare a SQL JOIN with the equivalent Prisma query.

## Chapter Summary

You learned Prisma relationships, @relation, one-to-one, one-to-many, many-to-many, select, include, nested queries, nested writes, and how these map to SQL JOINS.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 10: Prisma Studio

## 10.1 What is Prisma Studio?

Prisma Studio is a browser-based GUI that lets you view and edit the data in your database using your Prisma schema. It is designed for development and debugging.

## 10.2 Starting Prisma Studio

Run:

```
npx prisma studio
```

Prisma starts a local web interface where you can browse all models defined in `schema.prisma`.

## 10.3 Exploring Data

Select a model from the sidebar to view all records. You can sort, search and inspect relationships between models.

## 10.4 Creating Records

Click the Add Record button, enter field values and save. Prisma validates data based on your schema constraints.

## 10.5 Editing Records

Click any cell to update its value. Save the changes and Prisma writes them to the database immediately.

## 10.6 Deleting Records

Select a record and delete it from the interface. Use caution because changes affect the actual database.

## 10.7 Prisma Studio vs pgAdmin vs psql

Prisma Studio: Prisma-focused GUI for development.

pgAdmin: Full PostgreSQL administration tool.

psql: Command-line interface for PostgreSQL.

Choose the tool based on the task.

## 10.8 When to Use Prisma Studio

- Verify CRUD operations.
- Inspect relationships.
- Quickly edit development data.
- Debug backend applications.

## Best Practices

Use Prisma Studio only for development and testing. Keep production changes controlled through your application or administrative workflows.

## Common Mistakes

Editing production data accidentally, forgetting schema changes require migrations, and expecting Studio to replace database administration tools.

## Interview Questions

1. What is Prisma Studio?
2. How do you start Prisma Studio?
3. When would you use Prisma Studio instead of pgAdmin?
4. Can Prisma Studio modify data?
5. Is Prisma Studio intended for database administration?

## Practice Exercises

1. Launch Prisma Studio.
2. View all users.
3. Create a new product.
4. Update an existing user.
5. Delete a test record.

## Chapter Summary

You learned how to launch Prisma Studio, browse models, create, update and delete records, compare it with pgAdmin and psql, and use it effectively during development.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 11: Raw SQL with Prisma

## 11.1 Why Raw SQL?

Prisma covers most CRUD operations, but sometimes complex reports, database-specific features or performance optimizations require writing SQL directly.

## 11.2 \$queryRaw

Use `$queryRaw` to execute `SELECT` queries that return rows.

Example:

```
const users = await prisma.$queryRaw`SELECT * FROM users`;
```

## 11.3 \$executeRaw

Use `$executeRaw` for `INSERT`, `UPDATE`, `DELETE` or `DDL` statements where the primary result is the number of affected rows.

Example:

```
await prisma.$executeRaw`UPDATE users SET active=true`;
```

## 11.4 Safe Parameterized Queries

Use template literals or supported parameter binding.

Example:

```
const email='rahul@gmail.com';
```

```
await prisma.$queryRaw`SELECT * FROM users WHERE email=${email}`;
```

Prisma safely parameterizes the value.

## 11.5 Unsafe Methods

`$queryRawUnsafe` and `$executeRawUnsafe` execute SQL from plain strings. Avoid them with untrusted input because they can introduce SQL Injection vulnerabilities.

## 11.6 When to Use Raw SQL

- Complex JOINS
- Window functions
- CTEs
- Database-specific features
- Optimized reporting queries

## 11.7 SQL vs Prisma

Prisma CRUD is easier to read and maintain. Raw SQL provides maximum flexibility but requires deeper SQL knowledge and careful security practices.

## 11.8 Best Practices

Prefer Prisma methods first. Use raw SQL only when necessary. Always parameterize user input. Keep complex SQL documented and tested.

## Common Mistakes

Using unsafe methods with user input, mixing raw SQL everywhere instead of using Prisma, ignoring SQL Injection risks, and forgetting to validate inputs.

## Interview Questions

1. What is the difference between `$queryRaw` and `$executeRaw`?
2. When should raw SQL be preferred over Prisma methods?
3. Why are the Unsafe methods dangerous?
4. How does Prisma help prevent SQL Injection?
5. Give one example where raw SQL is useful.

## Practice Exercises

1. Fetch all users using `$queryRaw`.
2. Update a record using `$executeRaw`.
3. Rewrite an unsafe query as a safe parameterized query.
4. List situations where Prisma CRUD is better than raw SQL.
5. Compare a Prisma `findMany()` query with its raw SQL equivalent.

## Chapter Summary

You learned when to use raw SQL with Prisma, the difference between `$queryRaw` and `$executeRaw`, safe versus unsafe methods, parameterized queries, and recommended best practices.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 12: Transactions

## 12.1 What is a Transaction?

A transaction is a group of database operations that are treated as one unit. Either every operation succeeds (COMMIT) or all changes are rolled back (ROLLBACK).

## 12.2 Why Transactions?

Transactions keep data consistent. They are essential when multiple queries depend on each other and partial updates would leave the database in an invalid state.

## 12.3 Real-world Examples

- Bank transfer: Debit one account and credit another.
- E-commerce: Create an order, reduce stock and record payment.
- Booking systems: Reserve seats and create tickets together.

## 12.4 Prisma \$transaction()

Sequential transaction:

```
await prisma.$transaction([
  prisma.user.create({ data: { name: 'Nikhil' } }),
  prisma.profile.create({ data: { bio: 'Developer' } })
]);
```

If one operation fails, Prisma rolls back the entire transaction.

## 12.5 Interactive Transactions

Example:

```
await prisma.$transaction(async (tx) => {
  const user = await tx.user.create({ data: { name: 'Nikhil' } });
  await tx.order.create({
    data: {
      product: 'Laptop',
      userId: user.id
    }
  });
});
```

Use interactive transactions when later queries depend on earlier results.

## 12.6 Rollback

If any operation inside the transaction throws an error, all previous operations in that transaction are undone automatically.

## 12.7 Error Handling

Wrap transactions in try...catch. Log errors appropriately and return a suitable HTTP response without exposing internal database details.

## 12.8 Best Practices

- Keep transactions short.
- Include only related operations.
- Avoid long-running business logic inside transactions.
- Use interactive transactions only when required.

## Common Mistakes

- Performing unrelated work inside one transaction.
- Forgetting error handling.
- Assuming every query automatically runs in the same transaction.
- Holding transactions open for too long.

## Interview Questions

1. What is a transaction?
2. Why are transactions important?
3. What is the difference between sequential and interactive transactions?
4. What happens if one query fails inside a transaction?
5. Give a real-world example where transactions are necessary.

## Practice Exercises

1. Create a transaction that inserts a user and profile.
2. Build a bank transfer example.
3. Write an interactive transaction using tx.
4. Force an error and observe rollback.
5. Explain why transactions prevent inconsistent data.

## Chapter Summary

You learned what transactions are, why they matter, how to use Prisma's `$transaction()` API, the difference between sequential and interactive transactions, rollback behavior, best practices and interview concepts.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 13: Express + Prisma CRUD APIs

## 13.1 Architecture

Client → Routes → Controllers → prisma.js → PostgreSQL

This layered architecture keeps responsibilities separate and makes applications easier to maintain.

## 13.2 Suggested Folder Structure

```
project/
├── controllers/
├── routes/
├── prisma/
│   └── prisma.js
├── app.js
└── server.js
```

## 13.3 prisma.js

Create one shared PrismaClient instance.

```
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();
export default prisma;
```

## 13.4 GET API

Controller:

```
const users = await prisma.user.findMany();
return res.status(200).json(users);
```

## 13.5 POST API

Read req.body, validate input and create a record:  
await prisma.user.create({ data:{ name,email } });  
Return HTTP 201.

## 13.6 PUT API

Read req.params.id and req.body.

Update using:

```
prisma.user.update({ where:{ id:Number(id) }, data:{ name,email } });
```

## 13.7 DELETE API

Delete using:

```
prisma.user.delete({ where:{ id:Number(id) } });
```

## 13.8 Validation & Error Handling

Validate request data before querying the database. Wrap controller logic in try...catch and return appropriate HTTP status codes.

## 13.9 Best Practices

- One shared PrismaClient.
- Thin routes.
- Controllers contain business logic.
- Parameter validation.
- Proper status codes.
- Reusable helper functions.

## Common Mistakes

Creating PrismaClient inside controllers, forgetting await, using findUnique() with non-unique fields, not converting route parameters when required, and exposing internal errors.

## Interview Questions

1. Why should there be one PrismaClient instance?
2. What is the flow of a GET request?
3. Where should validation occur?
4. Why keep routes thin?
5. Which status code is returned after creating a resource?

## Practice Exercises

1. Build GET /users.
2. Build POST /users.
3. Build PUT /users/:id.
4. Build DELETE /users/:id.
5. Organize the project using routes, controllers and prisma.js.

## Chapter Summary

You learned how to build a production-style Express backend using Prisma, organize the project into layers, implement CRUD APIs, validate input, handle errors and follow backend best practices.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 14: Advanced Prisma Queries

## 14.1 count()

Returns the number of matching records.

```
await prisma.user.count();
```

## 14.2 aggregate()

Performs calculations.

```
await prisma.order.aggregate({ _sum: { price: true }, _avg: { price: true },  
_max: { price: true }, _min: { price: true } });
```

## 14.3 groupBy()

Groups rows by one or more fields.

```
await prisma.order.groupBy({ by: ['status'], _count: { status: true } });
```

## 14.4 distinct

Removes duplicate values.

```
await prisma.user.findMany({ distinct: ['email'] });
```

## 14.5 Relation Filters

Filter using related records.

some, every, none

Example:

```
where: { orders: { some: { status: 'PENDING' } } }
```

## 14.6 Nested Filtering

Filter inside related models.

Example:

```
include: { orders: { where: { status: 'DELIVERED' } } }
```

## 14.7 Advanced Ordering

Sort by multiple fields.

```
orderBy: [{ role: 'asc' }, { createdAt: 'desc' }]
```

## 14.8 Performance

Select only required fields, use pagination, index frequently filtered columns, avoid unnecessary nested includes.

## Best Practices

Prefer Prisma APIs before raw SQL, combine filtering with pagination, use relation filters for readability.

## Common Mistakes

Fetching unnecessary data, forgetting indexes, deeply nested queries without need, using include everywhere.

## Interview Questions

1. What does count() return?
2. Difference between aggregate() and groupBy()?
3. What is distinct?
4. Explain relation filters: some, every, none.
5. How do you optimize Prisma queries?

## Practice Exercises

1. Count all users.
2. Find average order price.
3. Group orders by status.
4. Return distinct emails.
5. Find users with at least one pending order.

## Chapter Summary

You learned advanced Prisma query capabilities including count, aggregate, groupBy, distinct, relation filtering, nested filtering, advanced ordering and query optimization.

# The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 3: Prisma ORM

Chapter 15: Prisma Interview Guide & Final Project

## 15.1 Prisma Revision Sheet

Core Topics:

- schema.prisma
- Models & Migrations
- Prisma Client
- CRUD
- Filtering & Sorting
- Pagination
- Relations
- Prisma Studio
- Raw SQL
- Transactions
- Express + Prisma
- Advanced Queries

## 15.2 Frequently Asked Interview Questions

Common questions:

1. What is Prisma ORM?
2. Difference between Prisma and raw SQL?
3. What is schema.prisma?
4. Why use Prisma Client?
5. Difference between findMany() and findUnique()?
6. What are migrations?
7. Explain select vs include.
8. What is Prisma Studio?
9. When would you use \$queryRaw?
10. What are transactions?
11. Difference between update() and updateMany()?
12. What is drift detection?
13. Why create only one PrismaClient instance?
14. Explain take and skip.
15. How do Prisma relations work?

## 15.3 Prisma vs SQL

Prisma:

- Type-safe
- Less boilerplate
- Faster development

SQL:

- Maximum control
- Better for very complex queries
- Requires manual query writing

## 15.4 Prisma vs Mongoose

Prisma:

- SQL databases
- Schema-first

- Generated client

Mongoose:

- MongoDB
- Document model
- ODM instead of ORM

## 15.5 Final Project

Build a User & Order Management API.

Features:

- Authentication
- CRUD for Users
- CRUD for Orders
- Relations
- Search
- Pagination
- Transactions
- Validation
- Error handling

Architecture:

Client

↓

Routes

↓

Controllers

↓

Prisma Client

↓

PostgreSQL

## 15.6 Best Practices

- One PrismaClient instance
- Parameter validation
- Thin controllers
- Keep secrets in .env
- Commit migrations
- Use select for minimal data
- Use transactions when needed

## 15.7 Common Mistakes

- Multiple PrismaClient instances
- Forgetting migrations
- Ignoring relations
- Using Unsafe raw SQL
- Returning unnecessary fields
- Missing error handling

## 15.8 Self Assessment

You should now be able to:

- Design Prisma schemas
- Create migrations
- Build CRUD APIs

- Work with relations
- Use pagination and filtering
- Execute raw SQL safely
- Build production-ready Express + Prisma applications

## Congratulations!

You have completed Volume 3 and the complete SQL → PostgreSQL → Prisma ORM learning path.

Continue by building real-world projects such as:

- E-commerce Backend
- Social Media API
- Chat Application
- Learning Management System
- CRM Dashboard

Consistent practice is the key to mastering backend development.